自然な TypeScript から JSON にするビルドツールを作るために工夫したこと

2024/07/10 kansai.ts #7

小原 一哉

自己紹介

- 小原 一哉 (こはら かずや)
- ウェブエンジニア
- フェンリル株式会社
- X: @KoharaKazuya

前回 (kansai.ts #6) で「最強の設定記述言語はすでにある」という発表をしました。

→ その中でチラッと話した OpenAPI Code の実現方法について紹介します。

OpenAPI Code については知らないと思いますが、課題を説明してから実現方法について話すので大丈夫です

やりたいこと

以下のような TypeScript を JSON ファイルに変換したい

```
// user.ts
export default { openapi: "3.1.0" };

// paths/pets.ts
import Pets from "#/components/schemas/Pets";
export default {
  get: {
    responses: {
      "200": {
        content: {
          "application/json": {
            schema: Pets,
          },
        },
      },
    },
  },
};

// components/schemas/Pets.ts
export default { type: "array" };
```

ポイント

- TypeScript で記述 = 関数の評価など普通のプログラムと同様にスクリプトを実行する必要がある
- 最終的に一つの JSON ファイルになる定義の一部が別ファイルに分割されていて、import 文で読み込んでいる → これと OpenAPI の `{ "$ref": "#/…" }` という特殊な書き方が対応している
- ファイルパスで埋め込む JSON の位置を指定している

必要なこと

1. `index.ts` を実行して、export default のオブジェクトを JSON ファイルにする
2. `paths/` ディレクトリ以下に存在するファイルをパスに対応する JSON 中の位置に埋め込む
3. `import X from "#/components/schemas/X";` を `{ $ref: "#/components/schemas/X" }` に変換する

実現方法 1

> `index.ts` を実行して、export default のオブジェクトを JSON ファイルにする

→ シンプルに実行 & 出力

1. `index.ts` (を JS に変換したもの) を import するコードを生成する
    - ビルド用の一時ディレクトリは `node_modules/.cache/` 以下に生成している (これはおそらく広く使われている慣習)
2. 1 でインポートしたオブジェクトを JSON に書き込むコードを生成する
3. Node.js に 1, 2 を実行させる

実現方法 2

> `paths/` 以下に存在するファイルをパスに対応する JSON 中の位置に埋め込む

→ 特定パス以下をスキャンし、コードを生成する

1. Rollup でバンドル＆コード変換する
   (バンドルしなくてもいいが、コード変換に便利な Rollup を使う)
2. 1 の Rollup 処理中に特定パス以下をスキャンして、ファイルがあればコードを生成し、埋め込む
   - JSON の `"paths": {}` のオブジェクトの中に埋め込む
   - 該当ファイルを import するコードを生成する
   - 見つけたパスをオブジェクトのキー、import した変数を値にする

Rollup: JavaScript をバンドルするやつ。Vite も内部で使ってる

実現方法 3

> `import X from "#/components/schemas/X";` を
> `{ $ref: "#/components/schemas/X" }` に変換する

→ `#/` から始まるパスで import されているモジュールを置き換える

1. Rollup でバンドル＆コード変換する
2. 1 の Rollup 処理中に `#/` から始まるパスのモジュールを置き換える
    - `export default { $ref: '#/…' };` のような文字列を生成する
    - Rollup で対象のモジュールの中身をすべて置き換える

ソースコード中では TypeScript による import として扱われるが、出力した JSON では `{ "$ref": "#/…" }` となる＆ `$ref` の先の実際の定義は `paths/` 以下の埋め込みと同様の方法で JSON に埋め込まれている。

加えて、他にもごにょごにょっとやると、TypeScript で共通関数がかけて、ファイル分割できて、型チェックできて、自動補完できて、エディタでリンクジャンプできる OpenAPI Document を出力するビルドツールを作ることができる。

まとめ

- OpenAPI Code を実現するために工夫したポイントを紹介した
- `node_modules/.cache/` が一時ディレクトリとして使える
- コードを変換するために Rollup が便利